



St. Francis Institute of Technology

(Engineering College)

An Autonomous Institute, Affiliated to University of Mumbai

NAAC A+ Accredited | CMPN, EXTC, INFT NBA Accredited | ISO 9001:2015 Certified

Department of Artificial Intelligence and Machine Learning

Academic Year: 2025-2026 **Term:** Even (Jan. 2026 – Jun. 2026) **Class / Branch:** SE – AIML

Semester: IV **Course:** Web Programming Lab. (AI4VS_LR4)

Date of Assignment: 17/03/2026 **Date of Submission:** 30/03/2026

Pre-Lab Exercises for Experiment-11

Part A: Understanding NoSQL and MongoDB

Task 1: Conceptual Preparation

1. What is NoSQL?

Ans. NoSQL (Not Only SQL) is a type of database that stores and manages data in non-tabular formats such as documents, key-value pairs, graphs, or wide-column stores. It is designed to handle large volumes of structured, semi-structured, and unstructured data with high scalability and flexibility.

2. How is NoSQL different from SQL databases?

Ans. NoSQL databases differ from SQL databases in structure, schema, and scalability. SQL databases store data in tables with a fixed schema, while NoSQL databases store data in flexible formats like documents or key-value pairs with a dynamic schema. SQL databases mainly handle structured data and scale vertically, whereas NoSQL databases can handle structured, semi-structured, and unstructured data and scale horizontally. SQL uses a standard query language, while NoSQL does not have a single standard query language.

3. What is MongoDB?

Ans. MongoDB is a NoSQL, document-oriented database that stores data in the form of JSON-like documents and provides high scalability and performance.

4. What is a document in MongoDB?

Ans. A document in MongoDB is a basic unit of data stored in JSON-like format consisting of key-value pairs.

5. What is a collection?

Ans. A collection in MongoDB is a group of related documents, similar to a table in relational databases.

6. What is BSON?

Ans. BSON (Binary JSON) is a binary-encoded format used by MongoDB to store documents, supporting additional data types and enabling efficient storage and retrieval.

7. What are the advantages of MongoDB?

Ans.

- a. Flexible schema
- b. High performance
- c. Horizontal scalability
- d. Handles large volumes of data
- e. High availability through replication
- f. Easy to use JSON-like format

Part B: MongoDB Practice (Without Express)

Task 2: Install and Explore MongoDB

1. Install MongoDB locally OR create a free account in **MongoDB Atlas**.
2. Create a new database named:
 - practiceDB

```
test> use practiceDB
switched to db practiceDB
```

3. Create a collection:
 - students

Perform the following operations:

1. Insert 3 student records manually.

```
practiceDB> db.students.insertMany([
|   { name: "Angel", age: 20, course: "AI" },
|   { name: "Sania", age: 19, course: "CA" },
|   { name: "Kenn", age: 21, course: "Web" }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69c9022b42c69013523682d1'),
    '1': ObjectId('69c9022b42c69013523682d2'),
    '2': ObjectId('69c9022b42c69013523682d3')
  }
}
```

2. View inserted records.

```
practiceDB> db.students.find().pretty()
[
  {
    _id: ObjectId('69c9022b42c69013523682d1'),
    name: 'Angel',
    age: 20,
    course: 'AI'
  },
  {
    _id: ObjectId('69c9022b42c69013523682d2'),
    name: 'Sania',
    age: 19,
    course: 'CA'
  },
  {
    _id: ObjectId('69c9022b42c69013523682d3'),
    name: 'Kenn',
    age: 21,
    course: 'Web'
  }
]
```

3. Delete one record.

```
practiceDB> db.students.deleteOne({ name: "Angel" })
{ acknowledged: true, deletedCount: 1 }
practiceDB> |
```

4. Update one record.

```
practiceDB> db.students.updateOne(
|   { name: "Kenn" },
|   { $set: { age: 20 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
practiceDB> db.students.find().pretty()
[
  {
    _id: ObjectId('69c9022b42c69013523682d2'),
    name: 'Sania',
    age: 19,
    course: 'CA'
  },
  {
    _id: ObjectId('69c9022b42c69013523682d3'),
    name: 'Kenn',
    age: 20,
    course: 'Web'
  }
]
```

Part C: Mongoose Familiarization

Task 3: Setup Basic Backend with Mongoose

1. Create a new Node.js project.

```
C:\Users\Angel>mkdir web11

C:\Users\Angel>cd web11

C:\Users\Angel\web11>npm init -y
Wrote to C:\Users\Angel\web11\package.json:
```

2. Install:

- express
- mongoose

```
C:\Users\Angel\web11>npm install express mongoose
added 82 packages, and audited 83 packages in 10s
```

3. Connect to MongoDB database.

index.js

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://127.0.0.1:27017/practiceDB')
  .then(() => {
    console.log("MongoDB Connected Successfully");
  })
  .catch((err) => {
    console.log("Connection Error:", err);
  });
```

4. Confirm successful connection in terminal.

```
C:\Users\Angel\web11>node index.js
MongoDB Connected Successfully
```

5. Questions:

- a. What happens if the connection string is wrong?

Ans. If the connection string is incorrect, Mongoose will fail to establish a connection with the MongoDB database. This results in a connection error, and the application will display an error message in the terminal. As a result, database operations such as reading or writing data cannot be performed.

- b. What is the purpose of mongoose?

Ans. Mongoose is an Object Data Modeling (ODM) library for MongoDB in Node.js. It is used to simplify database operations by providing a structured schema-based approach, allowing developers to define models, validate data, and interact with the database more easily compared to using MongoDB directly.

Task 4: Create Schema and Model

1. Create a schema with fields:
 - name
 - email
 - age
2. Create a model using the schema.
3. Insert one record using the model.
4. Retrieve all records from collection.

index.js

```
const mongoose =
require('mongoose');
// Connect to DB
mongoose.connect('mongodb://127.0
.0.1:27017/practiceDB')
  .then(() => console.log("MongoDB
Connected"))
  .catch(err => console.log(err));
// Create Schema
const studentSchema = new
mongoose.Schema({
  name: String,
  email: String,
  age: Number
});
// Create Model
const Student =
```

```
mongoose.model('Student',
studentSchema);
const insertData = async () => {
  const student = new Student({
    name: "Angel Xavier",
    email:
"angelxavier905@gmail.com",
    age: 19
  });
  await student.save();
  console.log("Data Inserted");
  // Retrieve all records
  const data = await
Student.find();
  console.log(data);
};
insertData();
```

OUTPUT

```
C:\Users\Angel\web11>node index.js
MongoDB Connected
Data Inserted
[
  {
    _id: new ObjectId('69c908d3a00b102d12c28031'),
    name: 'Angel Xavier',
    email: 'angelxavier905@gmail.com',
    age: 19,
    __v: 0
  }
]
```

Observation:

- Is the data stored in MongoDB?
Ans. Yes, the data is successfully stored in MongoDB as it can be retrieved using the `find()` method, confirming that the insertion operation was successful.
- Does schema restrict data structure?
Ans. Yes, the schema defines the structure for new documents by specifying the fields and their data types. However, MongoDB itself is schema-flexible, so existing documents with different structures can still exist in the same collection.

Part D: REST API Practice

Task 5: Create Basic REST APIs

1. Create the following routes:

POST → Add user

GET → Retrieve all users

index.js

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
app.use(express.json());
// Connect DB
mongoose.connect('mongodb://127.0.0.1:27017/practiceDB')
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.log(err));
// Schema
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});
// Model
```

```
const User = mongoose.model('User', userSchema);
// POST → Add user
app.post('/users', async (req, res) => {
  const user = new User(req.body);
  await user.save();
  res.status(201).json(user);
});
// GET → Retrieve all users
app.get('/users', async (req, res) => {
  const users = await User.find();
  res.status(200).json(users);
});
// Server
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

OUTPUT

```
C:\Users\Angel\web11>node index.js
Server running on port 3000
MongoDB Connected
```

2. Test using: Postman POST

My Collection > Post data

POST http://localhost:3000/users

Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "Angel Xavier",
3   "email": "angelxavier905@gmail.com",
4   "age": 20
5 }
```

Body 201 Created • 31 ms • 349 B • Save Response

JSON Preview Visualize

```
1 {
2   "name": "Angel Xavier",
3   "email": "angelxavier905@gmail.com",
4   "age": 20,
5   "_id": "69c90ef25394e1c5e6ec090d",
6   "__v": 0
7 }
```

GET

GET http://localhost:3000/users

Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "Angel Xavier",
3   "email": "angelxavier905@gmail.com",
4   "age": 20
5 }
```

Body 200 OK • 52 ms • 346 B • Save Response

JSON Preview Visualize

```
1 [
2   {
3     "_id": "69c90fdaebeafa89b2c2ac3c",
4     "name": "Angel Xavier",
5     "email": "angelxavier905@gmail.com",
6     "age": 20,
7     "__v": 0
8   }
9 ]
```

3. Verify:

a. Status codes

Ans. The POST request returned a **201 (Created)** status code, indicating that a new user was successfully added to the database. The GET request returned a **200 (OK)** status code, indicating that the request was successful and data was retrieved correctly.

b. JSON response format

Ans. The API responses were returned in proper JSON format. The POST request returned a JSON object containing the inserted user details, while the GET request returned an array of JSON objects representing all users in the database.

Part E: React API Practice

Task 6: Fetch Data in React

1. Create a simple React app.

```
C:\Users\Angel\web11>node index.js
Server running on port 5000
MongoDB Connected
```

Backend

```
C:\Users\Angel\web11-react>npm start

> web11-react@0.1.0 start
> react-scripts start
```

React

2. Create a component:

- UserList

UserList.js

```
import React, { useEffect, useState }
from 'react';

function UserList() {
  const [users, setUsers] =
useState([]);
  useEffect(() => {
    fetch('http://localhost:5000/users')
      .then(res => res.json())
      .then(data => setUsers(data))
      .catch(err => console.log(err));
  }, []);
  return (
    <div>
```

```
<h2>User List</h2>
<ul>
  {users.map((user) => (
    <li key={user._id}>
      {user.name} - {user.email}
      - {user.age}
    </li>
  ))}
</ul>
</div>
);
}
export default UserList;
```

3. Fetch data from backend GET API.

App.js

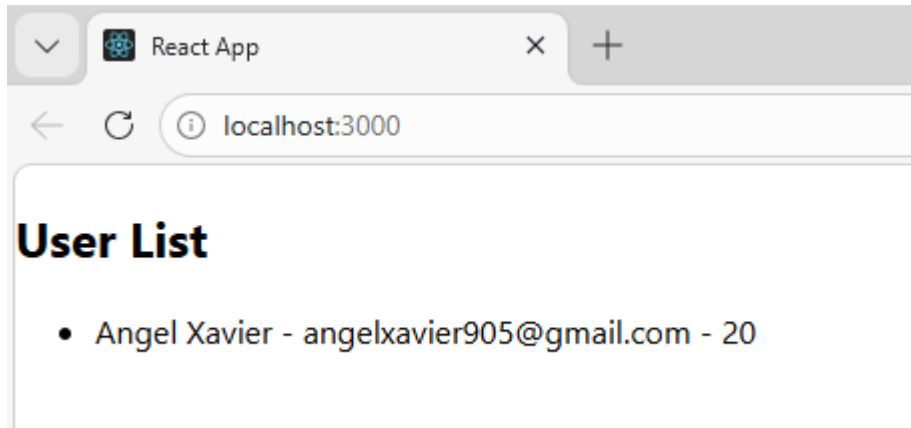
```
import React from 'react';
import UserList from './UserList';
function App() {
  return (
    <div>
```

```
      <UserList />
    </div>
  );
}
export default App;
```

4. Store data in state.

5. Display data using:

- List OR Table



index.js

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
// Middleware
app.use(express.json());
// ★ CORS FIX (no installation
needed)
app.use((req, res, next) => {
  res.header("Access-Control-Allow-Origin", "*");
  res.header("Access-Control-Allow-Headers", "Origin, X-Requested-With, Content-Type, Accept");
  next();
});
// Connect MongoDB
mongoose.connect('mongodb://127.0.0.1:27017/practiceDB')
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.log(err));
// Schema
const userSchema = new
mongoose.Schema({
```

```
  name: String,
  email: String,
  age: Number
});
// Model
const User = mongoose.model('User',
userSchema);
// POST API
app.post('/users', async (req, res)
=> {
  const user = new User(req.body);
  await user.save();
  res.status(201).json(user);
});
// GET API
app.get('/users', async (req, res) =>
{
  const users = await User.find();
  res.status(200).json(users);
});
// Server
app.listen(5000, () => {
  console.log("Server running on
port 5000");
});
```


Observation:

1. Does the UI update after fetching data?

Ans. Yes, the UI updates automatically after fetching data because the fetched data is stored in the component's state, and any change in state triggers re-rendering of the UI.

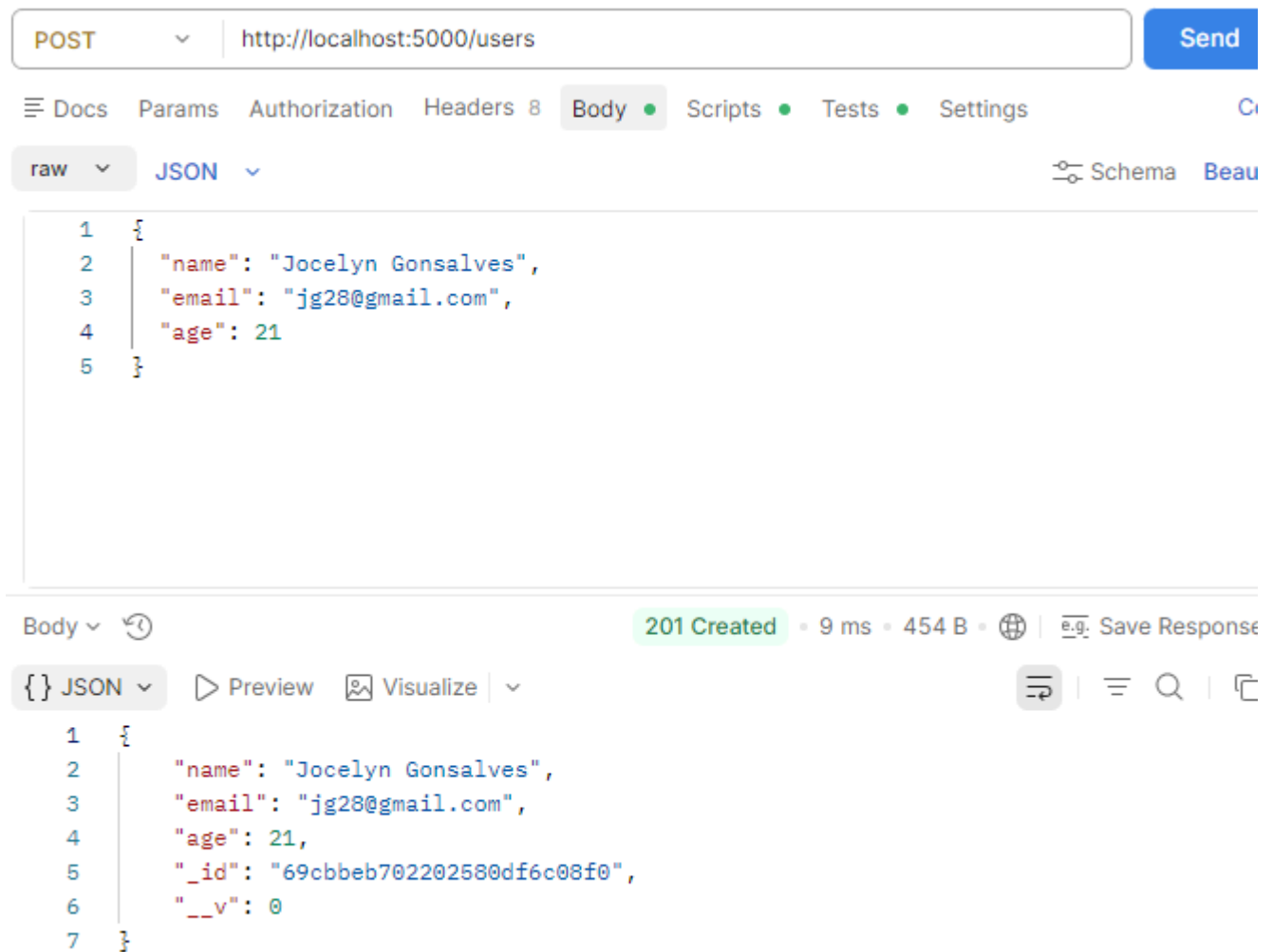
2. What happens if the backend is not running?

Ans. If the backend is not running, the fetch request fails and an error occurs. As a result, no data is displayed in the UI, and an error message may appear in the browser console.

Part F: Data Persistence Practice

Task 7: Restart Server Test

1. Add user using POST request.

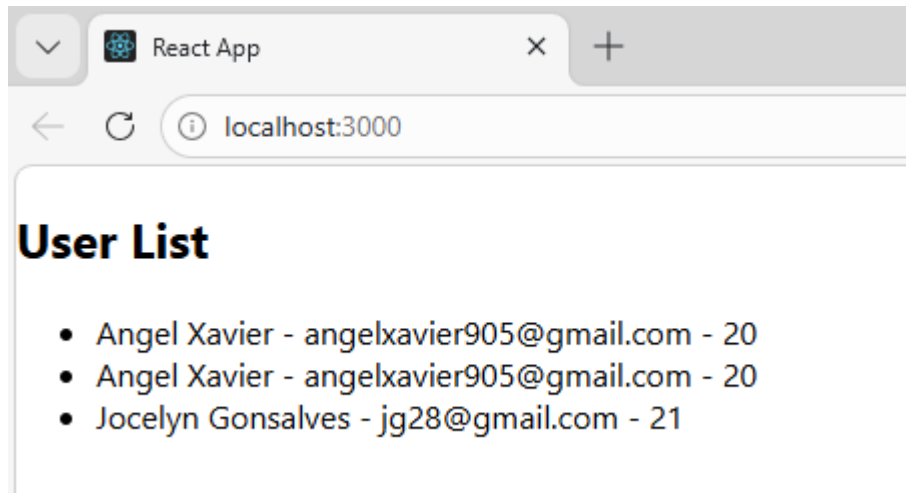


The screenshot shows a REST client interface. At the top, the method is set to POST and the URL is http://localhost:5000/users. The 'Body' tab is selected, showing a JSON payload: { "name": "Jocelyn Gonsalves", "email": "jg28@gmail.com", "age": 21 }. Below the request, the response is displayed with a status of 201 Created, a time of 9 ms, and a size of 454 B. The response body is a JSON object: { "name": "Jocelyn Gonsalves", "email": "jg28@gmail.com", "age": 21, "_id": "69cbb702202580df6c08f0", "__v": 0 }.

2. Restart backend server.

```
^C
C:\Users\Angel\web11>node index.js
Server running on port 5000
MongoDB Connected
```

3. Fetch users again.



Question:

1. Does data still exist? Why?

Ans. Yes, the data still exists after restarting the server because it is stored in the MongoDB database, which saves data permanently on disk. Restarting the backend server only restarts the application and does not affect the data stored in the database.

Task 8: Perform All CRUD Operations via API Activities:

1. Extend the backend to include:

- PUT → Update user by ID
- DELETE → Delete user by ID

UserList.js

```
import React, { useEffect, useState } from 'react';

function UserList() {
  const [users, setUsers] =
  useState([]);
  useEffect(() => {

  fetch('http://localhost:5000/users')
    .then(res => res.json())
    .then(data => setUsers(data))
    .catch(err => console.log(err));
  }, []);
  return (
    <div>
      <h2>User List</h2>
      <ul>
        {users.map((user) => (
          <li key={user._id}>
```

```
          {user.name} - {user.email}
          - {user.age}
        </li>
      )}
    </ul>
  </div>
  );
}
```

```
export default UserList;
```

App.js

```
import React from 'react';
import UserList from './UserList';

function App() {
  return (
    <div>
      <UserList />
    </div>
  );
}
```

```
export default App;
```

Index.js

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();

// Middleware
app.use(express.json());
// ★ CORS FIX
app.use((req, res, next) => {

  res.header("Access-Control-Allow-Origin", "*");

  res.header("Access-Control-Allow-Headers", "Origin, X-Requested-With, Content-Type, Accept");

  next();
});

// Connect MongoDB
mongoose.connect('mongodb://127.0.0.1:27017/practiceDB')
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.log("Connection Error:", err));

// Schema with validation
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true },
  age: { type: Number, min: 18 }
});

// Model
const User = mongoose.model('User', userSchema);

// ✓ POST → Create user
app.post('/users', async (req, res) => {
  try {
    const user = new User(req.body);
```

```
    await user.save();
    res.status(201).json(user);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// ✓ GET → Get all users
app.get('/users', async (req, res) => {
  try {
    const users = await User.find();
    res.status(200).json(users);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ✓ PUT → Update user by ID
app.put('/users/:id', async (req, res) => {
  try {
    const updatedUser = await User.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );
    if (!updatedUser) {
      return res.status(404).json({ message: "User not found" });
    }
    res.status(200).json(updatedUser);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// ✓ DELETE → Delete user by ID
app.delete('/users/:id', async (req, res) => {
  try {
    const deletedUser = await User.findByIdAndDelete(req.params.id);
```

```

    if (!deletedUser) {
        return res.status(404).json({
message: "User not found" });
    }
    res.status(200).json({ message:
"User deleted" });
} catch (err) {
    res.status(400).json({ error:
err.message });
}
}

// Server
app.listen(5000, () => {
    console.log("Server running on port
5000");
});

```

2. Test all four operations using Postman:
 - Create user (POST)

The screenshot shows the Postman interface for a POST request to `http://localhost:5000/users`. The request body is a JSON object with the following data:

```

{
  "name": "Jaden Borges",
  "email": "jb20@gmail.com",
  "age": 19
}

```

The response status is **201 Created**, with a response time of 113 ms and a body size of 448 B. The response body is a JSON object:

```

{
  "name": "Jaden Borges",
  "email": "jb20@gmail.com",
  "age": 19,
  "_id": "69dc5587043ff6e88acab39f",
  "__v": 0
}

```

- Retrieve users (GET)

GET

http://localhost:5000/users

Send

Docs

Params

Authorization

Headers 8

Body

Scripts

Tests

Settings

Cookies

raw

JSON

Schema

Beautify

1

{

Body

200 OK • 15 ms • 769 B • Save Response

{}

JSON

Preview

Visualize

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

]

"_id": "69cbbe8b02202580df6c08ee",

"name": "Angel Xavier",

"email": "angelxavier905@gmail.com",

"age": 20,

"__v": 0

},

{

"_id": "69cbbeb702202580df6c08f0",

"name": "Jocelyn Gonsalves",

"email": "jg28@gmail.com",

"age": 21,

"__v": 0

},

{

"_id": "69dc5587043ff6e88acab39f",

"name": "Jaden Borges",

"email": "jb20@gmail.com",

"age": 19,

"__v": 0

}

- Update user (PUT)

The screenshot shows a REST client interface with the following details:

- Request:**
 - Method: **PUT**
 - URL: `http://localhost:5000/users/69c90fdaebecfa89b2c2ac3c`
 - Body (JSON):

```
{
  "age": 23
}
```
- Response:**
 - Status: **200 OK** (21 ms, 454 B)
 - Body (JSON):

```
{
  "_id": "69c90fdaebecfa89b2c2ac3c",
  "name": "Angel Xavier",
  "email": "angelxavier905@gmail.com",
  "age": 23,
  "__v": 0
}
```

- Delete user (DELETE)

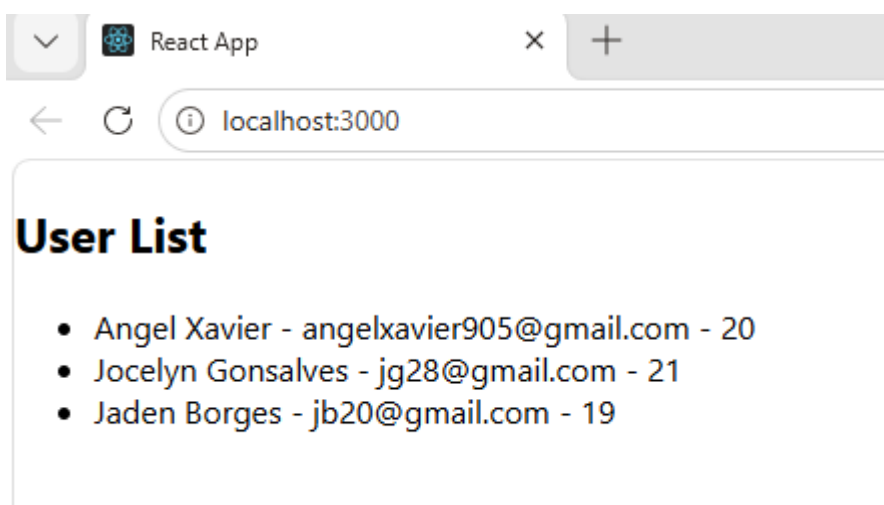
The screenshot shows a REST client interface with the following details:

- Request:**
 - Method: **DELETE**
 - URL: `http://localhost:5000/users/69c90fdaebecfa89b2c2ac3c`
- Response:**
 - Status: **200 OK** (11 ms, 371 B)
 - Body (JSON):

```
{
  "message": "User deleted"
}
```

3. Verify changes in MongoDB database.

```
already on db practiceDB
practiceDB> db.users.find().pretty()
[
  {
    _id: ObjectId('69cbbe8b02202580df6c08ee'),
    name: 'Angel Xavier',
    email: 'angelxavier905@gmail.com',
    age: 20,
    __v: 0
  },
  {
    _id: ObjectId('69cbb702202580df6c08f0'),
    name: 'Jocelyn Gonsalves',
    email: 'jg28@gmail.com',
    age: 21,
    __v: 0
  },
  {
    _id: ObjectId('69dc5587043ff6e88acab39f'),
    name: 'Jaden Borges',
    email: 'jb20@gmail.com',
    age: 19,
    __v: 0
  }
]
```



React App

localhost:3000

User List

- Angel Xavier - angelxavier905@gmail.com - 20
- Jocelyn Gonsalves - jg28@gmail.com - 21
- Jaden Borges - jb20@gmail.com - 19

Observations to Record:

1. What status codes are returned for each operation?

Ans. POST (Create) → 201 (Created)

GET (Retrieve) → 200 (OK)

PUT (Update) → 200 (OK)

DELETE (Delete) → 200 (OK)

2. What happens if an invalid ID is used?

Ans. If an invalid ID is used, the request fails and an error is returned. The API responds with 400 (Bad Request) or 404 (Not Found), and no changes are made to the database.

3. Does the database reflect changes immediately?

Ans. Yes, the database reflects changes immediately. Any create, update, or delete operation is instantly stored in MongoDB and can be verified using GET requests or MongoDB tools.

Task 9: Implement Basic Validation in Mongoose

Activities:

1. Modify schema to:

- Make name required
- Make email required
- Add minimum age validation

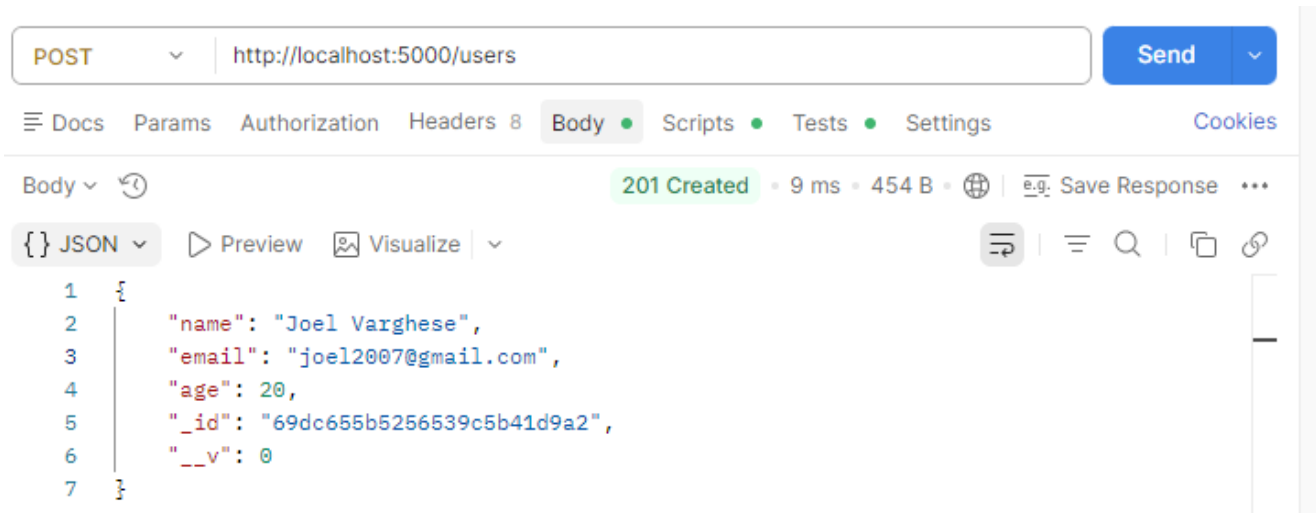
index.js

```
//  Schema with validation
```

```
const userSchema = new mongoose.Schema({  
  name: { type: String, required: true },  
  email: { type: String, required: true },  
  age: { type: Number, min: 18 }  
});
```

2. Try inserting:

- Valid data



The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:5000/users
- Buttons:** Send, Docs, Params, Authorization, Headers 8, Body, Scripts, Tests, Settings, Cookies
- Body Tab:** Selected, showing a JSON response.
- Status:** 201 Created
- Time:** 9 ms
- Size:** 454 B
- JSON Body:**

```
{  
  "name": "Joel Varghese",  
  "email": "joel2007@gmail.com",  
  "age": 20,  
  "_id": "69dc655b5256539c5b41d9a2",  
  "__v": 0  
}
```


- Invalid data (missing fields or incorrect age)

The screenshot shows a Postman interface for a POST request to `http://localhost:5000/users`. The request body is a JSON object:

```
1 {
2   "name": "Joel
   Varghese",
3   "email":
   "joel2007@gmail.
   com",
4   "age": 15
5 }
```

The response is a **400 Bad Request** with a status of 12 ms and 451 B. The response body is a JSON error message:

```
1 {
2   "error": "User validation failed: age: Path `age` (15) is less than minimum
   allowed value (18)."
```

3. Observe error messages.

Questions to Answer:

1. Does MongoDB allow invalid data without schema?
Ans. Yes, MongoDB allows invalid data as it is schema-less.
2. How does Mongoose validation improve data quality?
Ans. It ensures only valid and structured data is stored by enforcing rules.
3. Where are validation errors displayed?
Ans. In the API response (Postman) as JSON error messages.

Task 10: Handle Errors Gracefully

Activities: 1. Add error handling in:

- Database connection
- POST route
- GET route

index.js

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
app.use(express.json());
```

```
// CORS
```

```
app.use((req, res, next) => {
  res.header("Access-Control-Allow-Origin", "*");
  res.header("Access-Control-Allow-Header
```

```

rs", "Origin, X-Requested-With,
Content-Type, Accept");
    next();
  });
// ✓ DB connection with error
handling
mongoose.connect('mongodb://127.0.0.1:
27017/practiceDB')
.then(() => console.log("MongoDB
Connected"))
.catch(err => console.log("DB
Connection Error:", err));
// Schema
const userSchema = new
mongoose.Schema({
  name: { type: String, required: true
},
  email: { type: String, required:
true },
  age: { type: Number, min: 18 }
});
const User = mongoose.model('User',
userSchema);
// ✓ POST with error handling
app.post('/users', async (req, res) =>
{
  try {
    const user = new User(req.body);
    await user.save();
    res.status(201).json(user);
  } catch (err) {
    res.status(400).json({ error:
err.message });
  }
});
// ✓ GET with error handling
app.get('/users', async (req, res) =>
{
  try {
    const users = await User.find();
    res.status(200).json(users);
  } catch (err) {
    res.status(500).json({ error:

```

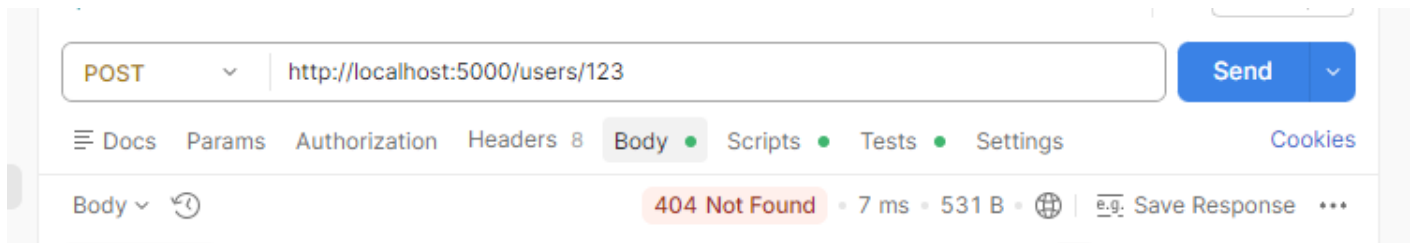
```

err.message });
  }
});
// ✓ PUT with error handling
app.put('/users/:id', async (req, res)
=> {
  try {
    const user = await
User.findByIdAndUpdate(req.params.id,
req.body, { new: true });
    if (!user) {
      return res.status(404).json({
message: "User not found" });
    }
    res.status(200).json(user);
  } catch (err) {
    res.status(400).json({ error:
err.message });
  }
});
// ✓ DELETE with error handling
app.delete('/users/:id', async (req,
res) => {
  try {
    const user = await
User.findByIdAndDelete(req.params.id);
    if (!user) {
      return res.status(404).json({
message: "User not found" });
    }
    res.status(200).json({ message:
"User deleted" });
  } catch (err) {
    res.status(400).json({ error:
err.message });
  }
});
// Server
app.listen(5000, () => {
  console.log("Server running on port
5000");
});

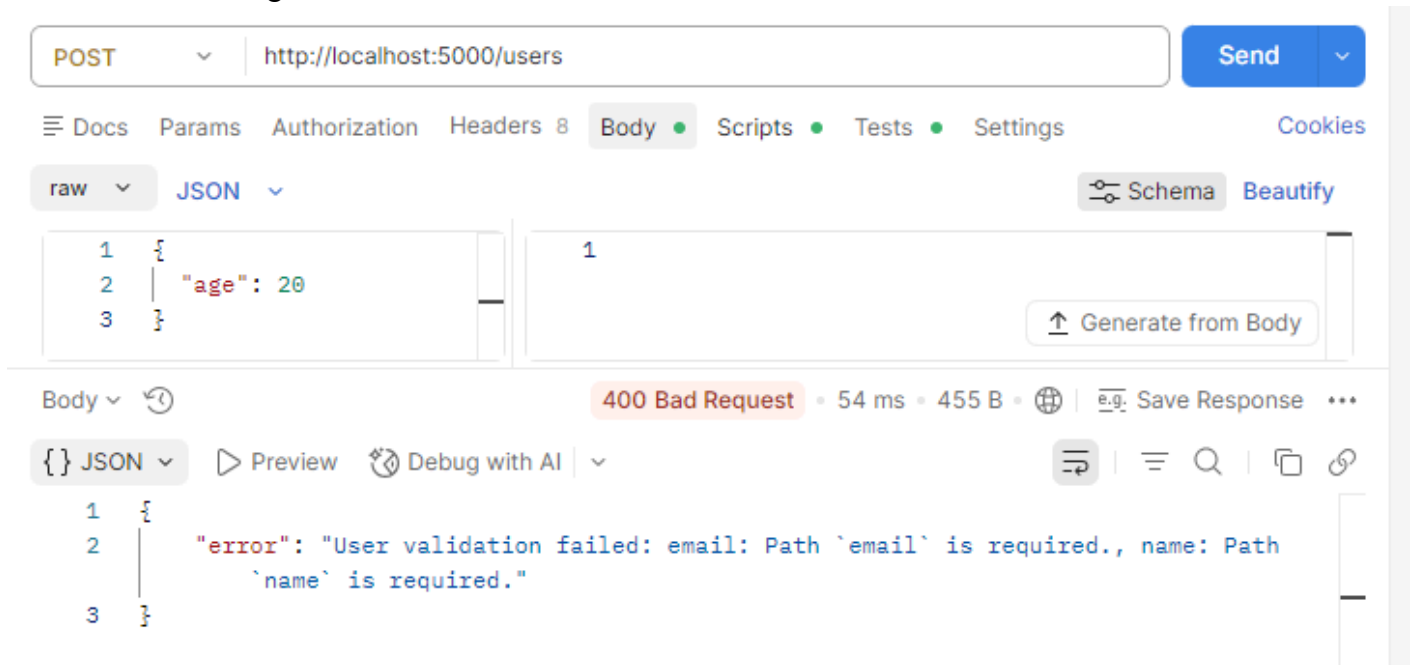
```

2. Send incorrect requests from Postman:

- Invalid ID



- Missing fields



3. Return proper:

- Status codes (400, 404, 500)
- JSON error messages

Observations:

1. What status code is returned for bad requests?
Ans. Bad requests return **400 (Bad Request)** status code, indicating that the request contains invalid or missing data.
2. What happens if the MongoDB server is stopped?
Ans. If the MongoDB server is stopped, the API is unable to connect to the database and returns a **500 (Internal Server Error)**. Data cannot be fetched or modified.
3. How does proper error handling improve API reliability?
Ans. Proper error handling improves API reliability by preventing crashes, returning appropriate status codes, and providing clear error messages, ensuring the system behaves consistently even when errors occur.